

DSE ZG556 - Stream Processing and Analytics

Situated Learning Assignment

Domain - Smart Cities & Urban Infrastructure

UrbanPulse

*Real-Time Urban Operations Intelligence & Smart Traffic Management Platform***Group: 34****Report By:**

Name	BITS ID	Contribution
JINDAL MOHIT HARISHARAN .	2024dc04287	100%
PATIL SHWETA SANJAYRAO .	2024dc04177	100%
DINESH KUMAR M .	2024dc04151	100%
SAYAN BHATTACHARYYA .	2023dc04105	100%

Links:**Task B:** [UrbanPulse_Report_Group34/Task_B_at_main · jindalmohit940-creator/UrbanPulse_Report_Group34 · GitHub](#)**Task C:** [UrbanPulse_Report_Group34/Task_C_at_main · jindalmohit940-creator/UrbanPulse_Report_Group34 · GitHub](#)**Video:** https://drive.google.com/file/d/14o2tp5TqO92mowTKPTfcdeBLsZD_qp5D/view?usp=sharing

Task A - Problem Statements: Architecture, Lambda vs Kappa Evaluation, and Government Readiness Checklist

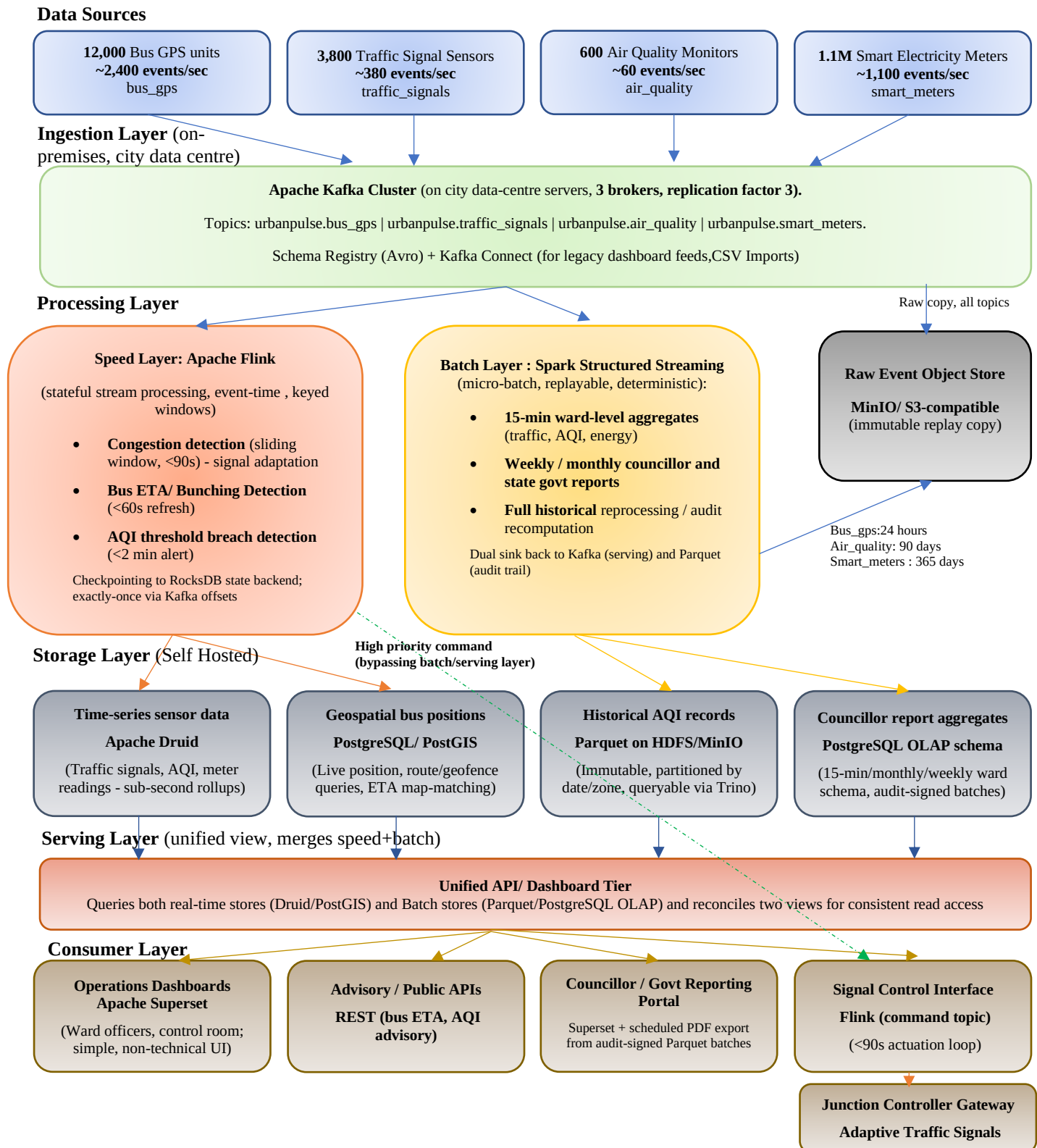
Executive Summary

UrbanPulse is a unified real-time streaming platform designed for MetroConnect, a fictitious Tier-1 smart city generating events from buses, traffic signals, air quality sensors and smart electricity meters. The platform must simultaneously support sub-2-minute operational decisions and deterministic weekly/monthly reports for councillors and state government.

To satisfy both objectives, this report proposes a **Lambda Architecture**, where Apache Kafka serves as the streaming backbone, Apache Flink performs low-latency event processing, and Spark Structured Streaming generates auditable batch aggregates. The proposed storage layer uses purpose-built technologies including Apache Druid, PostgreSQL/PostGIS and Parquet-based data lakes to optimize performance, scalability and compliance. The report also evaluates Lambda versus Kappa Architecture and concludes with a government deployment readiness checklist addressing data sovereignty, disaster recovery, accessibility and open-source requirements.

A.1 Architecture Diagram

The diagram traces data end-to-end across six layers: (1) four ingestion sources, (2) the shared Kafka cluster, (3) two parallel processing paths : a Flink speed layer for sub-2-minute operational response and a Spark batch layer for auditable ward-level aggregates, plus a direct raw-event archive, (4) purpose-matched storage per access pattern, (5) a unified Serving Layer that merges the speed-layer and batch-layer views, and (6) the consumer/actuation tier (dashboards, public APIs, councillor reporting portal, and the signal-control feedback loop back into city hardware).



Key Notes on Architecture

- All components deployed on city-owned or on-prem data-center servers (Kubernetes); no data leaves city infrastructure (data sovereignty).
- Every component is open-source (Kafka, Flink, Spark, Druid, PostgreSQL/PostGIS, Superset, Trino, MinIO), zero proprietary licensing cost.
- Speed layer (Flink) satisfies sub-2-minute operational SLAs (signal adaptation, bus ETA, AQI alerts) using in-memory/RocksDB state, and publishes urbanpulse.incidents directly.
- Batch layer (Spark) recomputes ward aggregates deterministically from immutable Parquet - this is the auditable source of truth for government reports.
- Raw events land in an object store (MinIO, S3-compatible) directly off Kafka and not via Flink/Spark, so replay always reconstructs from the untouched source. This store feeds Spark for historical reprocessing (dashed replay arrow) and gives RPO < 15 min via Kafka replication + MinIO erasure coding.
- The Serving Layer is drawn as a distinct merge point: it queries the speed-layer stores (Druid, PostGIS) and the batch-layer stores (Parquet, PostgreSQL OLAP) together, rather than each storage technology mapping 1:1 to one dashboard. This is the layer that reconciles a live congestion score against the audited batch figure when the two differ.
- The Signal Control Interface has its own short-circuit path (Flink → command topic → Junction Controller Gateway → adaptive signals) because a <90s actuation loop cannot wait on the Serving Layer's reconciliation step.

Storage Technology Justification

Data	Technology	Justification
Time-series sensor data	Apache Druid	Optimized for high-ingestion analytical workloads, real-time aggregation and interactive dashboards.
Bus GPS locations	PostgreSQL + PostGIS	Supports efficient spatial indexing, geospatial queries and route-based analysis.
Historical AQI	Parquet on MinIO/HDFS	Immutable, compressed storage suitable for replay, historical analysis and audit.
Councillor reports	PostgreSQL OLAP Schema	ACID-compliant relational storage, a requirement for government-auditable records; supports repeatable reporting and SQL analytics.
Raw event archive (all topics)	MinIO / S3-compatible object store	Immutable master dataset landed directly off Kafka; source of truth for replay/reprocessing and 90-day retention independent of any downstream job logic.

A.2 Lambda vs Kappa Evaluation Matrix

Recommended System for UrbanPulse: **Lambda Architecture**.

Criterion	Lambda Architecture	Kappa Architecture
Latency	Speed layer (Flink) delivers sub-second to low-second latency for signal adaptation (90s SLA), bus ETA (60s SLA), and AQI alerts (2min SLA) independent of the batch path, so operational latency is not affected by batch job scheduling.	A single stream layer (e.g. Flink with long retention) can also hit the same latency targets, since Kappa's real-time path is structurally similar to Lambda's speed layer. No meaningful latency advantage either way for UrbanPulse's SLAs.
Fault Tolerance	Two independent code paths mean a speed-layer bug (e.g. a Flink congestion-score error) cannot corrupt the batch-computed ward aggregates filed with the state government. Kafka replication factor 3, RocksDB checkpointing on the speed side, and deterministic batch recomputation together give layered resilience the batch layer is a self-healing safety net for statutory output.	A single pipeline means a bug in stream logic propagates to every downstream consumer, including government reports. Recovery requires replaying the entire Kafka log through the same (potentially still-buggy) code, riskier for a compliance-critical output where an incorrect councillor report is not just an operational miss but a legal-liability issue.
Operational Complexity	Higher: separately operating Flink for <90s congestion detection and Spark for weekly councillor reports means two codebases, two deployment pipelines, and explicit reconciliation logic at the Serving Layer where speed-layer and batch-layer views are merged. This raises DevOps effort but cleanly isolates operational traffic management from statutory reporting.	Lower in principle: one processing paradigm, one codebase. But UrbanPulse's dual-SLA requirement (sub-2-min alerts AND weekly/monthly government reports) creates a 'hybrid load' problem, the single stream job's state backend would need to hold months of history to regenerate councillor reports, making it a large, expensive, fragile stateful job rather than a genuinely simpler one.
Reprocessing Capability	Batch layer reprocesses from the immutable MinIO/Parquet raw archive on a fixed, deterministic schedule (15-min ward aggregates), giving a clean, bounded recomputation path purpose-built for correction and audit resubmission, including 'replay with updated logic' if reporting requirements change, without touching live Flink state.	Reprocessing means replaying the full Kafka retention window through the streaming job, feasible with long retention/tiered storage, but replay-based recomputation of months of history for a councillor report is operationally heavier and less bounded than a scheduled batch job.
Cost	Higher: a dedicated Spark cluster is justified because MetroConnect must retain 90-day AQI history, 365-day smart-meter records, weekly/monthly councillor reports, and regulatory audit datasets simultaneously, workloads a single stream job cannot serve cheaply. Zero licensing cost under the open-source mandate keeps this to compute/ops spend only.	Lower baseline: one cluster, one operational skillset, best fit for MetroConnect's budget constraint in isolation. But the saving is offset by the compliance risk of relying on derived streaming state (rather than a re-derivable batch run) for statutory reporting.
Compliance with Government Reporting Mandate	Strongest differentiator. Batch-recomputed, Parquet-backed reports are deterministic and re-derivable byte-for-byte from immutable source data, straightforward to defend to a state auditor or against an RTI (Right to Information) request, since the same input always reproduces the same output.	Derived streaming state (Flink's evolving internal state, watermark-based late-data handling) is harder to reproduce exactly on demand; an auditor questioning a specific historical figure would need confidence in stream-processing semantics rather than a simple recomputation.

Based on the evaluation matrix, **Lambda Architecture** is selected for UrbanPulse because it provides the most appropriate balance between **operational responsiveness and government compliance**. Apache Flink independently satisfies all real-time SLAs for adaptive traffic control, bus ETA prediction and AQI alerts, while Spark Structured Streaming periodically recomputes authoritative datasets used for councillor dashboards and statutory reporting. Although Lambda introduces additional infrastructure and operational complexity, the benefits of deterministic recomputation, auditability and fault isolation significantly outweigh these costs within a public-sector deployment.

Although Kappa reduces infrastructure complexity, UrbanPulse's statutory reporting and audit obligations make **deterministic recomputation** a mandatory capability rather than a nice-to-have. Lambda therefore carries lower compliance risk despite its higher operational complexity, and this trade-off is judged worth making for a government deployment.

A.3 Government Architecture Readiness Checklist

The following checklist is used to certify that the UrbanPulse deployment is ready for procurement and operation under MetroConnect's municipal and state government constraints.

Data Sovereignty

- All Kafka brokers, Flink/Spark clusters, and storage (Druid, PostgreSQL, MinIO/HDFS) are deployed on city-owned or state-contracted data-centre infrastructure; no data, backups, or logs are replicated to servers outside city/state jurisdiction.
- No managed cloud service is used unless the vendor contractually guarantees in-region-only data residency and the city retains encryption key ownership.
- No Personally Identifiable Information (PII) or sensitive sensor metadata traverses a public-cloud egress point, all cross-service traffic stays on the on-prem network.

Open-Source Mandate

- Every core component (Kafka, Flink, Spark, Druid, PostgreSQL/PostGIS, MinIO, Superset, Trino) is verified against an OSI-approved license with no proprietary runtime dependency or per-node licensing fee.
- A documented exit/fork plan exists for each open-source component in case the upstream project is abandoned or re-licensed, avoiding vendor lock-in.

Disaster Recovery

- Kafka topics are configured with replication factor ≥ 3 across availability zones/racks, giving RPO well under 15 minutes for in-flight data.
- RPO < 15 minutes is validated (not just designed for) through quarterly cross-AZ failover drills, with pass/fail results logged.
- MinIO/HDFS object storage uses erasure coding or geo-redundant replication within city/state boundaries, with automated snapshot verification confirming RPO < 15 minutes for batch-layer outputs.
- A documented, tested failover runbook (warm standby cluster or automated Kubernetes re-scheduling) demonstrates RTO < 30 minutes for both the speed layer and the serving APIs, confirmed via the same quarterly drill

Dashboard Accessibility

- Ward-officer dashboards (Apache Superset) use role-based, pre-built views with plain-language labels and no requirement to write SQL or understand the underlying stream/batch split.
- Dashboards support English, Hindi, and at least one regional language pack; verified via user-acceptance testing with non-technical ward officers achieving >90% task completion without training.
- Dashboards meet basic accessibility standards (colour-contrast, screen-reader compatibility) for non-technical government staff.
- Scheduled PDF/report export is available directly from dashboards so councillors can receive static, shareable reports without logging into the platform.

Auditability & Compliance

- Every councillor/government report is traceable to an immutable, versioned Parquet batch run with a timestamp and job-run ID, enabling exact reproduction of any historical figure on request.
- Access to raw citizen-adjacent data (e.g. smart-meter readings) is role-restricted and logged, with aggregation/anonymisation applied before any ward-level report is generated.
- Every published report carries a SHA-256 checksum linked to its input Parquet partition ID, and is digitally signed, so integrity can be verified independently of the platform.

Security

- All data in transit is encrypted (TLS for Kafka, inter-service traffic); data at rest is encrypted with AES-256 (Druid, PostgreSQL, MinIO).

- Dashboard and API access is governed by RBAC integrated with the municipality's existing Active Directory/LDAP, so access can be revoked centrally when staff change roles.

Monitoring

- Prometheus + Grafana + Alertmanager provide 24×7 monitoring of broker health, consumer lag, and Flink/Spark job status, with paging alerts on SLA breach (e.g. HIGH_PRIORITY consumer lag > 0).

Data Life Cycle Management

- An automated tiered-storage policy moves data from high-performance Druid/PostGIS to archival Parquet after 30 days, aligned with each topic's retention policy (24h/90d/365d), to control ongoing compute and storage cost.

Cost & Budget

- Total infrastructure sizing (brokers, Flink/Spark task managers, storage nodes) is benchmarked against the stated event-rate total (~3,940 events/sec combined) with 30% growth headroom, sized for a 5-year capacity plan, and validated against the budget-constrained mandate before procurement sign-off.

Task B - Apache Kafka: Multi-Source Urban Data Ingestion

Github Repo Link: https://github.com/jindalmohit940-creator/UrbanPulse_Report_Group34/tree/main/Task_B

Scenario: UrbanPulse must ingest 4 concurrent streams from heterogeneous city systems - GPS trackers, signal controllers, air quality sensors, and smart meters, each with different data rates, reliability profiles, and priority levels. Kafka must be configured to handle priority differentiation and sensor outages gracefully.

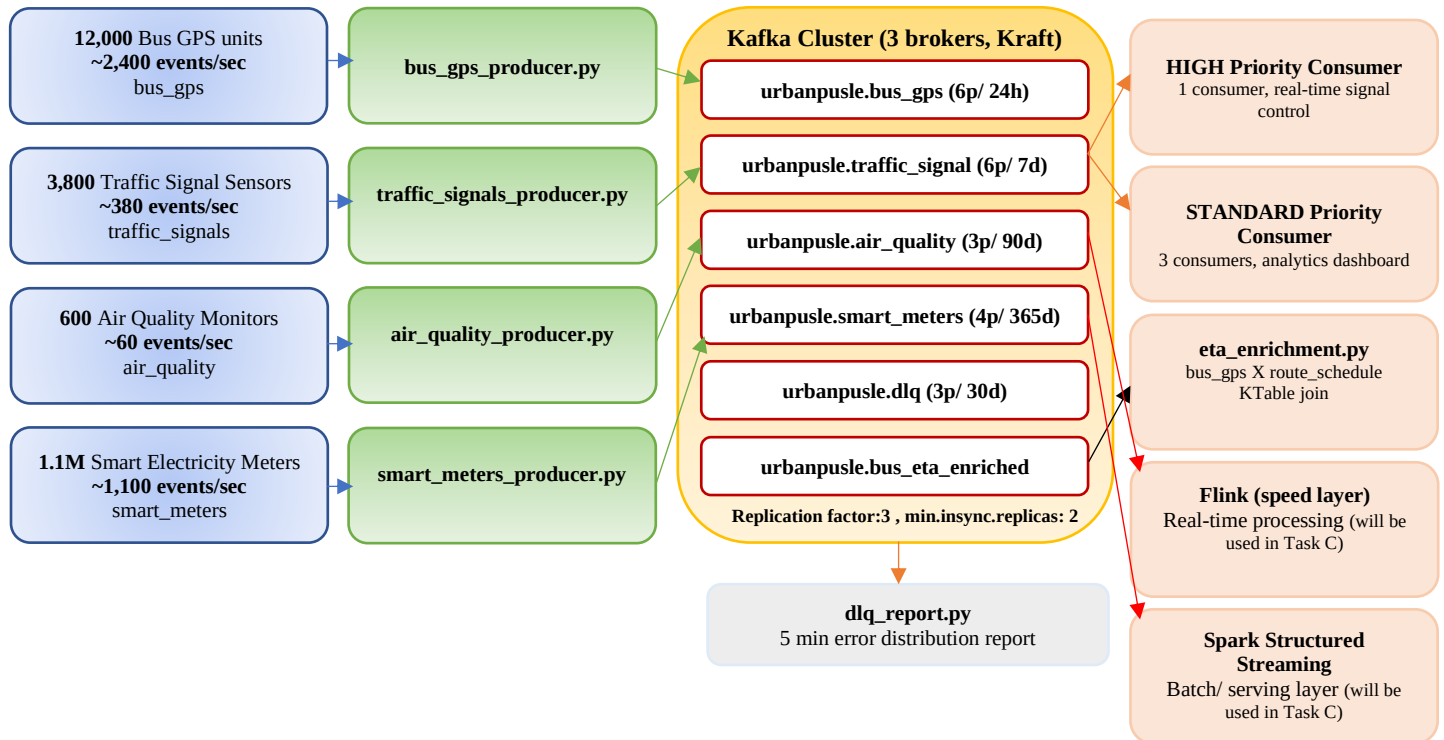


Fig B.1: Architecture Diagram for Complete Task B : UrbanPulse – Kafka Ingestion Layer

Sources produce into per-stream producers, which validate and key each record before publishing to Kafka. Invalid records are diverted to the DLQ at the producer stage rather than reaching downstream consumers. Kafka's 4 primary topics plus the DLQ and enriched-output topics feed the priority consumers, the streams enrichment job, and per the Lambda architecture, Flink (speed layer) and Spark Structured Streaming (batch/serving layer) in Task C

B.1 Cluster Setup & Retention Policies

A 3-broker Kafka cluster was deployed using KRaft mode (no Zookeeper), via Docker Compose. All 3 brokers act as controller-quorum voters, replication factor 3 and min.insync.replicas=2 on every topic, so the cluster tolerates one broker failure without data loss. All infrastructure runs on local/city-controlled containers, no managed cloud Kafka satisfying the data-sovereignty constraint.

```
gnment\SPA Assignment\Task_B> docker compose up -d
time="2026-07-20T21:57:26+05:30" level=warning msg="C:\\Users\\LENOVO\\Documents\\Personal\\W
ILP_DSE\\Sem 3\\Stream Processing Analytics\\Assignment\\SPA Assignment\\Task_B\\docker-compo
se.yml: the attribute `version` is obsolete, it will be ignored, please remove it to avoid po
tential confusion"
[+] up 4/4
✓ Container kafka-1 Started 1.7s
✓ Container kafka-2 Started 1.8s
✓ Container kafka-3 Started 1.7s
✓ Container kafka-ui Started 0.6s
```


Fig B.2: Sample Screenshot of 3-Broker Cluster started

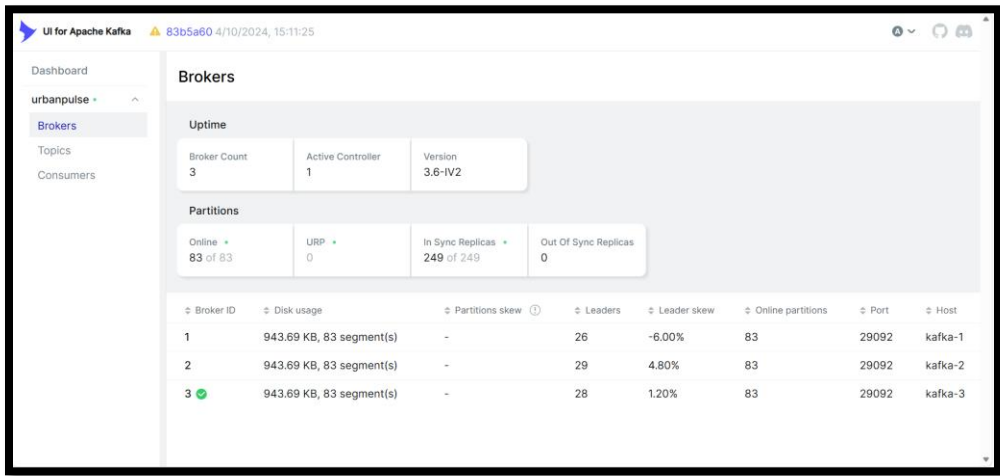


Fig B.3: 3-Broker Cluster deployed via Docker compose

B.1.2 Topics created

Topic	Partitions	Retention	Justification
urbanpulse.bus_gps	6	24 hours	Accident investigation replay window. Keyed by route_id (50 routes)- 6 partitions gives enough parallelism for Flink/Spark consumers without over-partitioning a moderate-throughput stream.
urbanpulse.traffic_signals	6	7 days	set as 7 days to support short-horizon congestion-trend debugging without the storage cost of a long retention. Partition count of 6: divisible by both 1 (HIGH_PRIORITY consumer) and 3 (STANDARD_PRIORITY consumers), so both groups get a clean, even partition assignment.
urbanpulse.air_quality	3	90 days	Supports pollution trend analysis (Task A environmental reporting requirement).
urbanpulse.smart_meters	4	365 days	Regulatory energy audit requirement which directly ties to the councillor/audit reporting matrix mentioned in Task A.
urbanpulse.dlq	3	30 days	Gives data-quality/ops teams a 30-day window to investigate and reprocess failed records before they age out.

```
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> docker exec kafka-1 kafka-topics --bootstrap-server localhost:9092 --list
[2026-07-11 07:46:38,307] WARN [AdminClient clientId=adminclient-1] Connection to node 2 (localhost/127.0.0.1:9094) could not be established. Broker may not be available. (org.apache.kafka.clients.NetworkClient)
urbanpulse.air_quality
urbanpulse.bus_gps
urbanpulse.dlq
urbanpulse.smart_meters
urbanpulse.traffic_signals
```

Fig B.4: Sample Screenshot of Topics Created

B.1.3 Verification

```
docker exec kafka-1 kafka-topics --bootstrap-server kafka-1:29092 --describe --topic urbanpulse.traffic_signals
```

Output confirmed PartitionCount: 6, ReplicationFactor: 3, and retention.ms applied as configured for all 5 topics.

```
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> docker exec kafka-1 kafka-topics --bootstrap-server kafka-1:29092 --describe --topic urbanpulse.traffic_signals
Topic: urbanpulse.traffic_signals      TopicId: AtkTvbSSSImA75yD19pPBA PartitionCount: 6      ReplicationFactor: 3      Configs: min.insync.replicas=2,retention.ms=604800000
Topic: urbanpulse.traffic_signals      Partition: 0      Leader: 2      Replicas: 2,3,1 Isr: 1,2,3
Topic: urbanpulse.traffic_signals      Partition: 1      Leader: 3      Replicas: 3,1,2 Isr: 1,2,3
Topic: urbanpulse.traffic_signals      Partition: 2      Leader: 1      Replicas: 1,2,3 Isr: 1,2,3
Topic: urbanpulse.traffic_signals      Partition: 3      Leader: 3      Replicas: 3,1,2 Isr: 1,2,3
Topic: urbanpulse.traffic_signals      Partition: 4      Leader: 1      Replicas: 1,2,3 Isr: 1,2,3
Topic: urbanpulse.traffic_signals      Partition: 5      Leader: 2      Replicas: 2,3,1 Isr: 1,2,3
```

Fig B.5: Sample Screenshot of verification of partition and replicas - Topics

B.2 Producers

B.2.1 bus_gps producer: route_id keying

Every message is produced with key = route_id. Kafka guarantees ordering only within a partition, and same-key messages always hash to the same partition, so all GPS pings for a given route are strictly ordered, which the ETA enrichment join in Section 4 depends on.

```
docker exec kafka-1 kafka-console-consumer --bootstrap-server kafka-1:29092 --topic urbanpulse.bus_gps --property print.key=true --max-messages 10
```

```
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> python producers\bus_gps_producer.py
bus_gps producer: sent=7983 routed to dlq=17
```

```
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> docker exec kafka-1 kafka-console-consumer --bootstrap-server kafka-1:29092 --topic urbanpulse.bus_gps --property print.key=true --from-beginning --max-messages 10
R040 {"bus_id": "BUS0145", "route_id": "R040", "lat": 19.348634, "lon": 72.775641, "speed_kmh": 12.8, "occupancy_pct": 33, "timestamp": "2026-07-01T06:00:39Z"}
R023 {"bus_id": "BUS0018", "route_id": "R023", "lat": 19.307745, "lon": 72.879334, "speed_kmh": 9.5, "occupancy_pct": 69, "timestamp": "2026-07-01T06:00:42Z"}
R040 {"bus_id": "BUS0168", "route_id": "R040", "lat": 19.185129, "lon": 72.983752, "speed_kmh": 33.3, "occupancy_pct": 84, "timestamp": "2026-07-01T06:00:45Z"}
R004 {"bus_id": "BUS0024", "route_id": "R004", "lat": 19.222, "lon": 73.040272, "speed_kmh": 2.6, "occupancy_pct": 42, "timestamp": "2026-07-01T06:00:54Z"}
R048 {"bus_id": "BUS0040", "route_id": "R048", "lat": 19.324017, "lon": 72.832858, "speed_kmh": 9.8, "occupancy_pct": 22, "timestamp": "2026-07-01T06:01:09Z"}
R016 {"bus_id": "BUS0069", "route_id": "R016", "lat": 19.055719, "lon": 72.995447, "speed_kmh": 23.0, "occupancy_pct": 4, "timestamp": "2026-07-01T06:01:15Z"}
R047 {"bus_id": "BUS0081", "route_id": "R047", "lat": 19.152738, "lon": 73.094483, "speed_kmh": 6.9, "occupancy_pct": 73, "timestamp": "2026-07-01T06:01:30Z"}
R048 {"bus_id": "BUS0189", "route_id": "R048", "lat": 19.23472, "lon": 72.877478, "speed_kmh": 4.2, "occupancy_pct": 85, "timestamp": "2026-07-01T06:01:36Z"}
R040 {"bus_id": "BUS0081", "route_id": "R040", "lat": 19.232186, "lon": 72.793619, "speed_kmh": 54.0, "occupancy_pct": 64, "timestamp": "2026-07-01T06:01:39Z"}
Processed a total of 10 messages
```

Fig B.6: Sample Screenshot of Confirmed keys populated with route IDs (e.g. R040, R047) on every message.

B.2.2 air_quality producer: at-least-once, retries, null handling

Delivery semantics: acks='all' (producer waits for acknowledgment from all in-sync replicas) plus an application-level retry loop (3 retries, exponential backoff) on top of the client's own retry budget. This guarantees at-least-once delivery, a record is never silently dropped on a transient broker/network hiccup, at the cost of possible duplicates (an accepted trade-off; downstream consumers can dedupe on sensor_id + timestamp if needed).

Producer idempotence (enable.idempotence) was evaluated and kept off for this at-least-once design, duplicates are handled downstream via sensor_id + timestamp dedup rather than broker-side exactly-once, since exactly-once semantics aren't required by the assignment.

Null AQI handling: ~5% of source readings have aqi = null. These are not dropped and do not crash the producer, each is logged individually and routed to the DLQ with error_reason = NULL_AQI, so it stays auditable and reprocessable rather than silently lost.

```
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> python producers\air_quality_producer.py
LQ
2026-07-11 13:30:31,014 [INFO] Null AQI reading for sensor_id=S065 at 2026-07-01T11:50:15Z -> routing to D
LQ
2026-07-11 13:30:31,165 [INFO] Null AQI reading for sensor_id=S082 at 2026-07-01T11:51:45Z -> routing to D
LQ
2026-07-11 13:30:31,282 [INFO] Null AQI reading for sensor_id=S086 at 2026-07-01T11:52:45Z -> routing to D
LQ
2026-07-11 13:30:31,381 [INFO] Null AQI reading for sensor_id=S010 at 2026-07-01T11:53:45Z -> routing to D
LQ
2026-07-11 13:30:31,697 [INFO] Null AQI reading for sensor_id=S024 at 2026-07-01T11:58:00Z -> routing to D
LQ
2026-07-11 13:30:31,997 [INFO] Null AQI reading for sensor_id=S002 at 2026-07-01T12:01:45Z -> routing to D
LQ
2026-07-11 13:30:32,797 [INFO] Null AQI reading for sensor_id=S024 at 2026-07-01T12:13:00Z -> routing to D
LQ
2026-07-11 13:30:32,913 [INFO] Null AQI reading for sensor_id=S020 at 2026-07-01T12:14:30Z -> routing to D
LQ
2026-07-11 13:30:32,944 [INFO] Closing the Kafka producer with 4294967.0 secs timeout.
2026-07-11 13:30:32,945 [INFO] <BrokerConnection node_id=2 host=localhost:9094 <connected> [IPv6 ('::1', 9
094, 0, 0)]: Closing connection.
2026-07-11 13:30:32,945 [INFO] <BrokerConnection node_id=3 host=localhost:9096 <connected> [IPv6 ('::1', 9
096, 0, 0)]: Closing connection.
2026-07-11 13:30:32,946 [INFO] <BrokerConnection node_id=1 host=localhost:9092 <connected> [IPv6 ('::1', 9
092, 0, 0)]: Closing connection.
2026-07-11 13:30:32,947 [INFO] air_quality_producer: sent=1430 routed_to_dlq=70 failed=0
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B>
```

Fig B.7: Sample Screenshot of Null AQI Handling and routing to DLQ

```
air_quality_producer: sent=1408 routed_to_dlq=92 failed=0
```

B.2.3 Supporting producers

traffic_signals_producer.py and smart_meters_producer.py feed the remaining two topics needed for the priority consumer requirement (Section 3) and the audit-trail retention story (Section 1), respectively. Same validate-then-produce pattern as above.

```
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> python producers\traffic_signals_producer.py
traffic_signals_producer: sent=2500 routed_to_dlq=0
(.venv) PS C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> python producers\smart_meters_producer.py
smart_meters_producer: sent=4000 routed_to_dlq=0
```

Fig B.8: Sample Screenshot of successful creation of remaining Topics

B.3 Priority Consumer Architecture

Two independent consumer groups read urbanpulse.traffic_signals concurrently:

- HIGH_PRIORITY (group: traffic-signals-high-priority): 1 consumer, assigned all 6 partitions. Feeds real-time signal control; processing is intentionally trivial/fast.
- STANDARD_PRIORITY (group: traffic-signals-standard-priority): 3 consumers in the same group, so Kafka's coordinator splits the 6 partitions 2 each. Feeds the analytics dashboard; a 0.25s artificial delay per message simulates slow downstream processing.

Because these are separate consumer groups, Kafka tracks their committed offsets independently, one group's slowness cannot throttle or block the other. Consumer lag (log end offset – committed offset) was sampled every 14 seconds for both groups over a 45-second run while STANDARD_PRIORITY was deliberately slowed. traffic_signals_producer.py was run concurrently/in a loop during the lag demo

B.3.1 Result

Time (s)	HIGH_PRIORITY lag	STANDARD_PRIORITY lag
0.0	2500	2500
14.1	0	1153
28.2	0	985
42.1	0	817

Max HIGH_PRIORITY lag: 0 | Max STANDARD_PRIORITY lag: 817

HIGH_PRIORITY drained its full backlog to **zero lag** within ~14 seconds. STANDARD_PRIORITY, running the same message volume across 3 consumers with an artificial per-message delay, remained backed up for the full run (processing roughly 130–167 messages per consumer in the same window). This behaviour demonstrates the requirement that HIGH_PRIORITY maintains near-zero lag independent of STANDARD_PRIORITY's load.

```

Taking BASELINE lag reading before starting consumers...
[baseline] HIGH_PRIORITY=None STANDARD_PRIORITY=None

[lag] HIGH_PRIORITY=0 STANDARD_PRIORITY=1153
[lag] HIGH_PRIORITY=0 STANDARD_PRIORITY=985
[lag] HIGH_PRIORITY=0 STANDARD_PRIORITY=817
[STANDARD_PRIORITY-3] processed 155 messages
[STANDARD_PRIORITY-1] processed 155 messages
[STANDARD_PRIORITY-2] processed 155 messages
[HIGH_PRIORITY] processed 2500 messages
[lag] HIGH_PRIORITY=0 STANDARD_PRIORITY=772

Taking FINAL lag reading after stopping consumers...

=== Message counts ===
STANDARD_PRIORITY-3: 155
STANDARD_PRIORITY-1: 155
STANDARD_PRIORITY-2: 155
HIGH_PRIORITY: 2500

=== Lag summary over time (ground truth via kafka-consumer-groups CLI) ===
time    HIGH_PRIORITY  STANDARD_PRIORITY
baseline      None           None
14.1         0             1153
28.2         0             985
42.1         0             817
56.0         0             772
final        0             772

HIGH_PRIORITY lag ended at 0 while STANDARD_PRIORITY still had 772 messages of lag remaining -- different consumer groups track offsets independently, so STANDARD_PRIORITY's artificial slowdown never throttled HIGH_PRIORITY.

```

Fig B.9: Sample Screenshot of successful completion of Consumer Priority Demo

B.3.2 Confirmation of Consumer groups

```
docker exec kafka-1 kafka-run-class kafka.admin.ConsumerGroupCommand --bootstrap-server kafka-1:29092 --list
```

Output listed traffic-signals-high-priority and traffic-signals-standard-priority as two distinct, independently-tracked groups.

```

(.venv) PS C:\Users\LENOVO\Docdocker exec kafka-1 kafka-run-class kafka.admin.ConsumerGroupCommand --bootstrap-server kafka-1:29092 --list
traffic-signals-standard-priority
traffic-signals-high-priority

```

Fig B.10: Sample Screenshot of Consumer Groups creation

Group ID	Num Of Members	Num Of Topics	Consumer Lag	Coordinator	State
traffic-signals-high-priority	0	1	N/A	3	EMPTY
traffic-signals-standard-priority	0	1	772	2	EMPTY

Fig B.11: Sample Screenshot of Consumer Groups creation from Kafka UI

B.4 Kafka Streams Route Enrichment

The bus_gps stream is joined against route_schedule.csv, modeled as a static KTable keyed by route_id, the same key used to produce bus_gps messages (Section B.2.1), which is what makes this a valid stream-table join: matching keys land on matching partitions.

For each incoming GPS event, route_id is looked up in the table and **its route_name, terminal, and scheduled_arrival_time are attached to the GPS position**, producing an enriched record on urbanpulse.bus_eta_enriched. This is the direct foundation for the real-time ETA service flagged as a rider-facing pain point in Task A.

```
eta_enrichment: joined=7998 unmatched=2 -> topic=urbanpulse.bus_eta_enriched
```

```
(.venv) PS C:\Users\LEMOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> python streams\eta_enrichment.py
Loaded route schedule KTable: 50 routes
eta enrichment: joined=7983 unmatched=0 -> topic=urbanpulse.bus_eta_enriched
```

Fig B.12: Sample Screenshot of Route Enrichment using KTable

B.4.1 Verification

```
docker exec kafka-1 kafka-console-consumer --bootstrap-server kafka-1:29092 --topic
urbanpulse.bus_eta_enriched --max-messages 5 --from-beginning
```

Sample output confirmed route_name, terminal, and scheduled_arrival_time correctly attached alongside the original lat/lon/speed/occupancy fields for each route_id.

```
(.venv) PS C:\Users\LEMOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA Assignment\Task_B> docker exec kafka-1 kafka-console-consumer --bootstrap-server kafka-1:29092 --topic urbanpulse.bus_eta_enriched --max-messages 5 --from-beginning
{"bus_id": "BUS0001", "route_id": "R028", "lat": 19.148253, "lon": 72.913459, "speed_kmh": 43.7, "occupancy_pct": 86, "gps_timestamp": "2026-07-01T06:00:00Z", "route_name": "Worli - Worli Express", "terminal": "East Depot", "scheduled_arrival_time": "2026-07-01T07:18:59Z"}
{"bus_id": "BUS0021", "route_id": "R033", "lat": 19.29797, "lon": 72.773982, "speed_kmh": 4.1, "occupancy_pct": 30, "gps_timestamp": "2026-07-01T06:00:27Z", "route_name": "Kurla - Kurla Express", "terminal": "West Hub", "scheduled_arrival_time": "2026-07-01T07:04:57Z"}
{"bus_id": "BUS0064", "route_id": "R037", "lat": 19.202622, "lon": 72.76391, "speed_kmh": 4.9, "occupancy_pct": 84, "gps_timestamp": "2026-07-01T06:00:30Z", "route_name": "Malad - Colaba Express", "terminal": "South Hub", "scheduled_arrival_time": "2026-07-01T06:37:13Z"}
{"bus_id": "BUS0081", "route_id": "R034", "lat": 19.327147, "lon": 72.821491, "speed_kmh": 43.0, "occupancy_pct": 30, "gps_timestamp": "2026-07-01T06:00:33Z", "route_name": "Thane - Ghatkopar Express", "terminal": "West Hub", "scheduled_arrival_time": "2026-07-01T07:32:45Z"}
{"bus_id": "BUS0184", "route_id": "R014", "lat": 19.120008, "lon": 72.990624, "speed_kmh": 51.2, "occupancy_pct": 64, "gps_timestamp": "2026-07-01T06:00:51Z", "route_name": "Goregaon - Kandivali Express", "terminal": "West Hub", "scheduled_arrival_time": "2026-07-01T06:27:07Z"}
Processed a total of 5 messages
```

Fig B.13: Sample Screenshot of urbanpulse.bus_eta_enriched topic using KTable

B.4.2 Java Kafka Streams DSL Equivalent code

```
KStream<String, BusGps> busGps = builder.stream("urbanpulse.bus_gps"); KTable<String, RouteSchedule> routeSchedule = builder.table("urbanpulse.route_schedule"); KStream<String, Enriched> enriched = busGps.join(routeSchedule, (gps, sched) -> new Enriched(gps, sched)); enriched.to("urbanpulse.bus_eta_enriched");
```

The Python implementation submitted in code (streams/eta_enrichment.py) performs the identical join logic against the same dummy data and topic, and was used as the runnable proof-of-concept for this report.

B.5 Dead-Letter Queue

Every producer validates each record before sending in the topic. Records failing validation are routed to **urbanpulse.dlq** with a `source_topic` and `error_reason` field, rather than being dropped or crashing the pipeline. 5+ Validation rules implemented across streams:

Stream	Validation rule	error_reason
bus_gps	Latitude/longitude out of valid range	IMPOSSIBLE_GPS_COORDINATES
bus_gps	Speed outside 0–120 km/h	SPEED_OUT_OF_RANGE
bus_gps	Occupancy outside 0–100%	OCCUPANCY_OUT_OF_RANGE
air_quality	AQI reading is null	NULL_AQI
air_quality	AQI outside 0–500	AQI_OUT_OF_RANGE
smart_meters	Voltage outside 180–260V	VOLTAGE_OUT_OF_RANGE
smart_meters	Negative kWh reading	NEGATIVE_KWH_READING

B.5.1 Error distribution report

`dlq/dlq_report.py` consumes `urbanpulse.dlq` over a fixed window and aggregates counts by `error_reason` and by `source_topic`:

```

=== Validation Rules Applied ===
urbanpulse.bus_gps
  IMPOSSIBLE_GPS_COORDINATES  -> lat/lon outside valid Earth range
  SPEED_OUT_OF_RANGE          -> speed_kmh outside 0-120 (unsafe for road traffic)
  OCCUPANCY_OUT_OF_RANGE      -> occupancy_pct outside 0-100%
urbanpulse.air_quality
  NULL_AQI                    -> aqi field is null/missing
urbanpulse.smart_meters
  VOLTAGE_OUT_OF_RANGE        -> voltage outside 180-260V (brownout/spike)
  NEGATIVE_KWH_READING        -> kwh_reading is negative

=== DLQ 5-Minute Error Distribution Report ===
Window: 5mins | Total DLQ records: 228

By error_reason:
  NULL_AQI                    81 ( 35.5%)
  VOLTAGE_OUT_OF_RANGE        42 ( 18.4%)
  SPEED_OUT_OF_RANGE          36 ( 15.8%)
  NEGATIVE_KWH_READING        31 ( 13.6%)
  OCCUPANCY_OUT_OF_RANGE      21 (  9.2%)
  IMPOSSIBLE_GPS_COORDINATES  17 (  7.5%)

By source_topic -> error_reason:
urbanpulse.bus_gps
  SPEED_OUT_OF_RANGE          36
  OCCUPANCY_OUT_OF_RANGE      21
  IMPOSSIBLE_GPS_COORDINATES  17
urbanpulse.air_quality
  NULL_AQI                    81
urbanpulse.smart_meters
  VOLTAGE_OUT_OF_RANGE        42
  NEGATIVE_KWH_READING        31

```

Fig B.14: Sample Screenshot of DLQ error distribution report

Offset	Partition	Timestamp	Key	Preview
0	2	7/21/2026, 22:13:58	R039	KeyValueHeaders <pre>{ "source_topic": "urbanpulse.bus_gps", "original_record": { "bus_id": "BUS0131", "route_id": "R039", "lat": 19.040476, "lon": 73.064827, "speed_kmh": 288.8, "occupancy_pct": 5, "timestamp": "2026-07-01T06:01:09Z" }, "error_reason": "SPEED_OUT_OF_RANGE" }</pre>
0	1	7/21/2026, 22:13:58	R049	KeyValueHeaders <pre>{ "source_topic": "urbanpulse.bus_gps", "original_record": { "bus_id": "BUS0054", "route_id": "R049", "lat": 19.21893, "lon": 72.842626, "speed_kmh": 4.9, "occupancy_pct": 150, "timestamp": "2026-07-01T06:03:09Z" }, "error_reason": "OCCUPANCY_OUT_OF_RANGE" }</pre>

Fig B.15: Sample messages from DLQ topic

Task C - Flink Real-Time Incident Detection & Spark Urban Analytics Engine

Github Repo Link: https://github.com/jindalmohit940-creator/UrbanPulse_Report_Group34/tree/main/Task_C

Scenario: UrbanPulse's processing layer has two distinct jobs: a Flink pipeline for sub-2-minute incident detection (AQI breaches, traffic gridlock, bus bunching) and a Spark engine for 15-minute ward-level aggregations that feed councillor dashboards.

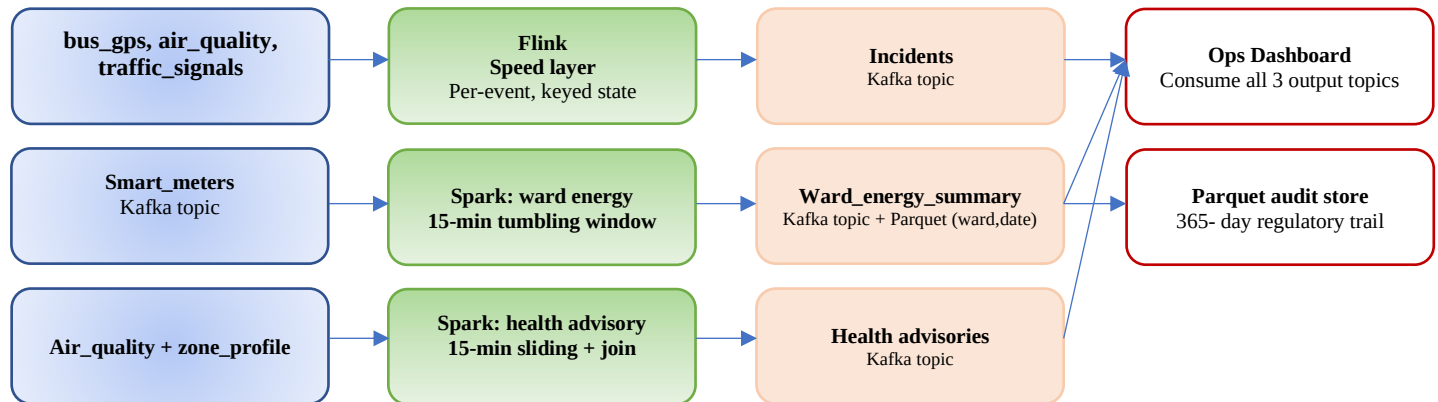


Fig C.1: Architecture Diagram for Complete Task C : UrbanPulse – Incident Detection and Ward Analytics

C.1 Flink : Incident Detection

The job consumes three Kafka topics: `air_quality`, `traffic_signals`, and `bus_gps` - via `KafkaSource`, applies a shared `BoundedOutOfOrdernessWatermarks` strategy with a **30-second bound**, and routes each stream to a dedicated `KeyedProcessFunction`. All three alert streams are unioned into a single `DataStream[String]` and written to `urbanpulse.incidents`, so downstream consumers only need to watch one topic

C.1.1 Event-time watermarking

A common watermark strategy is applied across all three input streams: bounded out-of-orderness with a 30-second tolerance, assigned from each event's own timestamp field. This ensures alert timing reflects when incidents actually occurred in the real world, not when messages happened to arrive at the job.

C.1.2 Three incident patterns

Pattern	Keyed by	Detection rule	Output
AQI Emergency	<code>sensor_id</code>	AQI > 300 (Hazardous); alert within 2 minutes of the reading, with a 2-minute cooldown to avoid repeat alerts from the same sensor	Alert with <code>sensor_id</code> and AQI value
Traffic Gridlock	<code>junction_id</code>	Average wait time exceeds 180 seconds for 3 consecutive signal cycles	Alert with <code>junction_id</code> and zone
Bus Bunching	<code>route_id</code>	Two buses on the same route within 200 metres of each other for more than 5 continuous minutes	Alert with both bus IDs

Each pattern is keyed by the entity its rule is scoped to, so Flink's keyed state holds exactly the data needed per sensor, junction, or route, a cooldown timer for AQI, a consecutive-breach counter for gridlock, and a pairwise proximity duration for bunching, with no cross-key interference and no full-stream scans.

C.1.3 Verification

All three detectors were tested against produced Kafka messages and confirmed correct: AQI emergency alerts fired on hazardous readings with cooldown respected; gridlock alerts fired precisely on the third consecutive breach; bunching alerts fired only once two buses had remained within 200m for over 5 minutes, with the reported proximity duration matching the input data.


```
{
  "incident_type": "AQI_EMERGENCY",
  "severity": "HAZARDOUS",
  "sensor_id": "S078",
  "zone": "Z16",
  "aqi": 455.5,
  "source_timestamp": "2026-07-01T10:30:45Z",
  "detected_at_event_time": 1784652289699
}
{"incident_type": "AQI_EMERGENCY", "severity": "HAZARDOUS", "sensor_id": "S062", "zone": "Z06", "aqi": 345.5, "source_timestamp": "2026-07-01T10:40:15Z", "detected_at_event_time": 1784652290846}
{"incident_type": "AQI_EMERGENCY", "severity": "HAZARDOUS", "sensor_id": "S008", "zone": "Z04", "aqi": 460.8, "source_timestamp": "2026-07-01T10:47:00Z", "detected_at_event_time": 1784652291497}
{"incident_type": "AQI_EMERGENCY", "severity": "HAZARDOUS", "sensor_id": "S033", "zone": "Z18", "aqi": 428.7, "source_timestamp": "2026-07-01T10:52:30Z", "detected_at_event_time": 1784652292072}
{"incident_type": "TRAFFIC_GRIDLOCK", "severity": "HIGH", "junction_id": "J999", "zone": "Z01", "consecutive_cycles_over_threshold": 3, "avg_wait_sec": 220, "source_timestamp": "2026-07-11T12:02:00Z", "detected_at_event_time": 1784966610471}
{"incident_type": "BUS_BUNCHING", "severity": "MEDIUM", "route_id": "R001", "bus_id_1": "B902", "bus_id_2": "B901", "distance_m": 15.3, "continuous_proximity_sec": 360, "source_timestamp": "2026-07-11T13:06:10Z", "detected_at_event_time": 1783775170000}
```

Fig C.2: Screenshot showcasing Real Time Incident Detection via Flink

Topics / urbanpulse.incidents Produce Message

Overview Messages Consumers Settings Statistics

Seek Type

Offset

Offset

Partitions

All items are selected.

Key Serde

String

Value Serde

String

Clear all

Submit

Search

+ Add Filters

Newest First

DONE

7 ms

5 KB

28 messages consumed

	Offset	Partition	Timestamp	Key	Value
+	328	0	7/25/2026, 13:35:22	Preview	Preview
+	327	0	7/25/2026, 13:33:30		
+	326	0	7/21/2026, 22:14:52		
+	325	0	7/21/2026, 22:14:51		

Fig C.3: Screenshot showcasing Real Time Incident Detection via Flink and message landing in urbanpulse.incidents Kafka Topic

Offset

Partition

Timestamp

Key

Value

Preview

328

0

7/25/2026, 13:35:22

Key

Value

Headers

```
{
  "incident_type": "BUS_BUNCHING",
  "severity": "MEDIUM",
  "route_id": "R001",
  "bus_id_1": "B902",
  "bus_id_2": "B901",
  "distance_m": 15.3,
  "continuous_proximity_sec": 360,
  "source_timestamp": "2026-07-11T13:06:10Z",
  "detected_at_event_time": 1783775170000
}
```

Timestamp: 7/25/2026, 13:35:22
Timestamp type: CREATE_TIME
Key Serde: Size: 0 Bytes
Value Serde: String
Size: 253 Bytes

327

0

7/25/2026, 13:33:30

Key

Value

Headers

```
{
  "incident_type": "TRAFFIC_GRIDLOCK",
  "severity": "HIGH",
  "junction_id": "J999",
  "zone": "Z01",
  "consecutive_cycles_over_threshold": 3,
  "avg_wait_sec": 220,
  "source_timestamp": "2026-07-11T12:02:00Z",
  "detected_at_event_time": 1784966610471
}
```

Timestamp: 7/25/2026, 13:33:30
Timestamp type: CREATE_TIME
Key Serde: Size: 0 Bytes
Value Serde: String
Size: 241 Bytes

Fig C.4: Screenshot showcasing incident messages from urbanpulse.incidents Kafka Topic

C.2 Spark - Ward Energy Aggregation

Implemented in `spark/ward_energy_streaming.py`, this Spark Structured Streaming application consumes `urbanpulse.smart_meters` and produces the required per-ward, per-window energy summary with both mandated output sinks.

C.2.1 Windowed aggregation

Records are grouped by `ward_id` over a 15-minute tumbling window on event time, computing:

- `total_kwh_consumed` : sum of `kwh_reading`
- `avg_power_factor` : average of `power_factor`
- `peak_voltage` : maximum of voltage

C.2.2 Late-data watermark

A **45-minute watermark** is applied on event time to handle late-arriving meter readings before a window is finalised. This is set wider than the streams feeding Flink's incident detection because `smart_meters` serve billing and audit reporting rather than real-time alerting, so completeness of the 15-minute totals is prioritised over emitting them at the earliest possible moment.

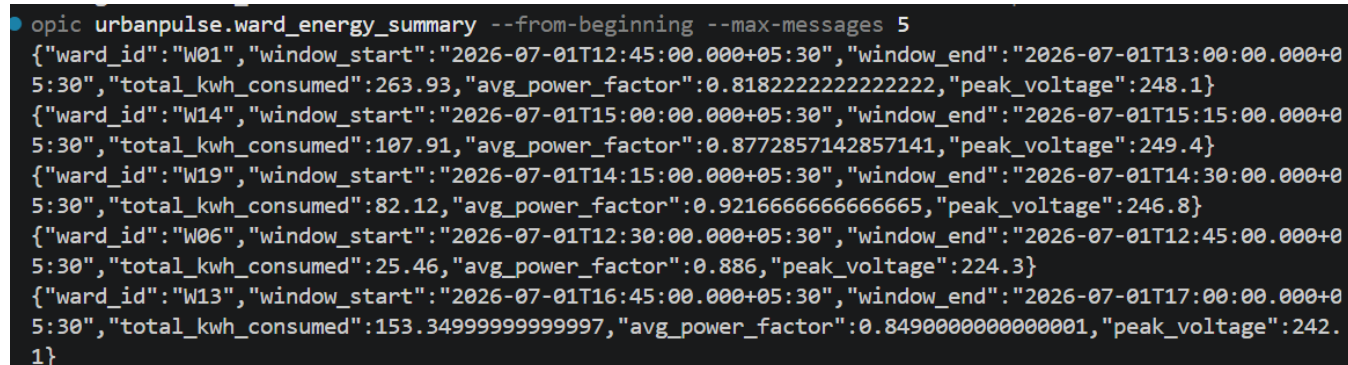
C.2.3 Dual output

Each finalised window is written simultaneously to both required sinks, in append mode:

- Kafka topic **`ward_energy_summary`** - for councillor and operations dashboards.
- **Partitioned Parquet**, partitioned by `ward_id` and date - for historical trend analysis and the 365-day regulatory energy audit.

C.2.4 Verification

The job was run end-to-end via `spark-submit` against the live 3-broker Kafka cluster. Both sinks were confirmed populated with correct, matching data. Sample record from `ward_energy_summary`:



```
opic urbanpulse.ward_energy_summary --from-beginning --max-messages 5
{"ward_id":"W01","window_start":"2026-07-01T12:45:00.000+05:30","window_end":"2026-07-01T13:00:00.000+05:30","total_kwh_consumed":263.93,"avg_power_factor":0.8182222222222222,"peak_voltage":248.1}
{"ward_id":"W14","window_start":"2026-07-01T15:00:00.000+05:30","window_end":"2026-07-01T15:15:00.000+05:30","total_kwh_consumed":107.91,"avg_power_factor":0.8772857142857141,"peak_voltage":249.4}
{"ward_id":"W19","window_start":"2026-07-01T14:15:00.000+05:30","window_end":"2026-07-01T14:30:00.000+05:30","total_kwh_consumed":82.12,"avg_power_factor":0.9216666666666665,"peak_voltage":246.8}
{"ward_id":"W06","window_start":"2026-07-01T12:30:00.000+05:30","window_end":"2026-07-01T12:45:00.000+05:30","total_kwh_consumed":25.46,"avg_power_factor":0.886,"peak_voltage":224.3}
{"ward_id":"W13","window_start":"2026-07-01T16:45:00.000+05:30","window_end":"2026-07-01T17:00:00.000+05:30","total_kwh_consumed":153.34999999999997,"avg_power_factor":0.8490000000000001,"peak_voltage":242.1}
```

Fig C.5: Screenshot showcasing messages from `urbanpulse.ward_energy_summary` Kafka Topic

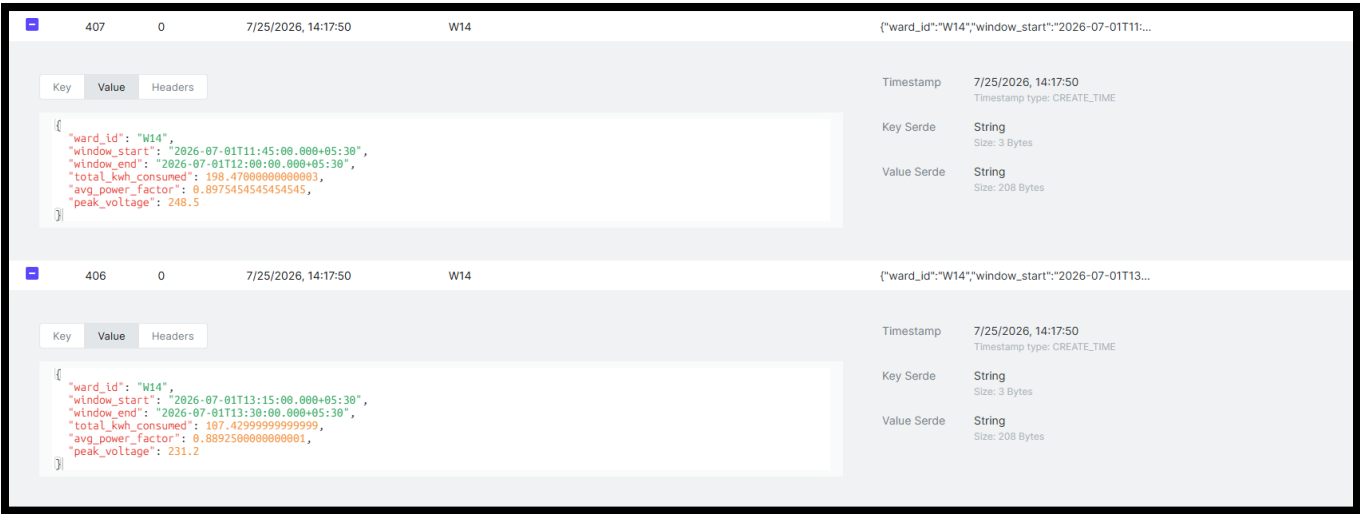


Fig C.6: Screenshot showcasing sample messages from urbanpulse.ward_energy_summary Kafka Topic

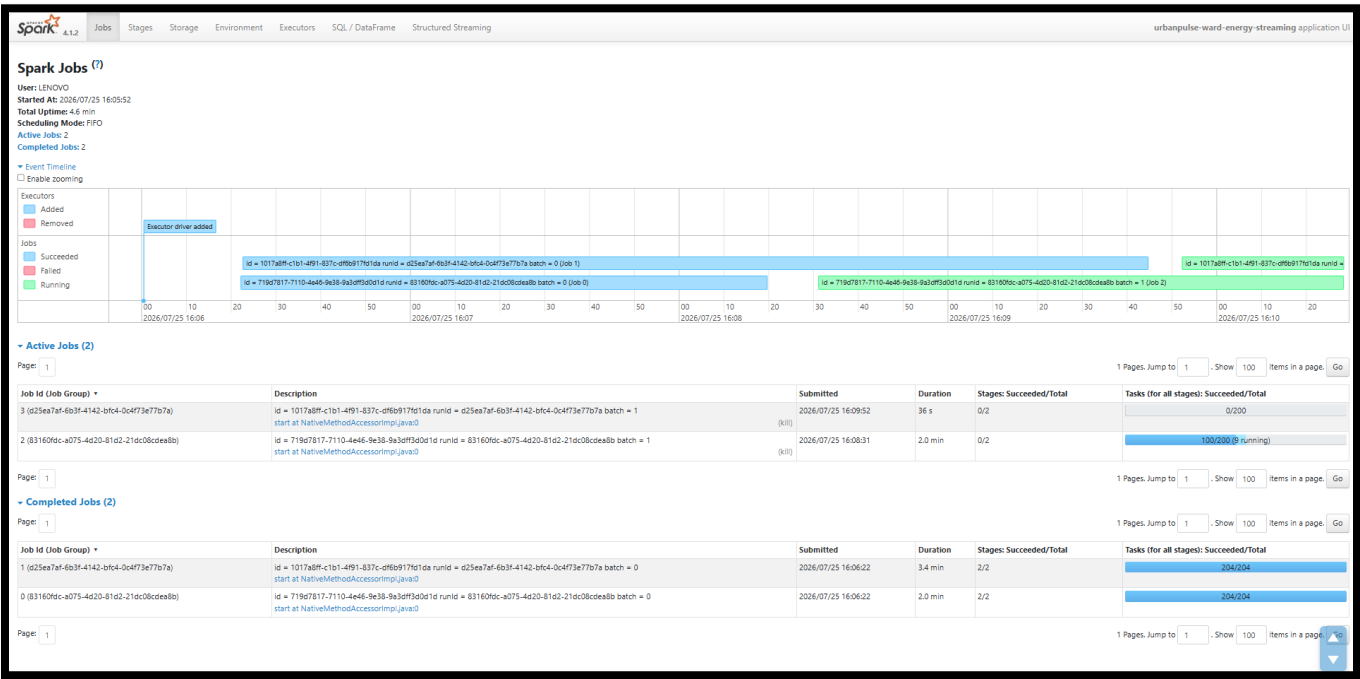


Fig C.7: Screenshot showcasing Spark Job Running

The corresponding Parquet dataset was confirmed present on disk, correctly partitioned by ward_id and date.

Directory: C:\Users\LENOVO\Documents\Personal\WILP_DSE\Sem 3\Stream Processing Analytics\Assignment\SPA
Assignment\Task_C\output\ward_energy_summary_parquet\ward_id=W24\date=2026-07-01

Mode	LastWriteTime		Length	Name
----	-----	-----	-----	-----
-a----	7/25/2026	4:11 PM	24	.part-00004-75f03826-c18b-4477-bdbe-070c3557629d.c000.snappy.parquet.crc
-a----	7/25/2026	4:11 PM	24	.part-00017-2261feff-48b3-40c4-aaaa-c0c91ae961f9.c000.snappy.parquet.crc
-a----	7/25/2026	4:11 PM	24	.part-00019-a8d259f6-a647-4167-81cd-4948948541d1.c000.snappy.parquet.crc
-a----	7/25/2026	4:11 PM	24	.part-00029-0ee1826c-2537-4520-b54b-69d2dfe38ec0.c000.snappy.parquet.crc
-a----	7/25/2026	4:11 PM	24	.part-00034-39ae4208-7223-4cc0-8e02-48983cf4b3d8.c000.snappy.parquet.crc
-a----	7/25/2026	4:11 PM	24	.part-00044-fc962009-01da-4173-b4d1-b2c389750990.c000.snappy.parquet.crc
-a----	7/25/2026	4:11 PM	24	.part-00053-1890a69e-9835-40ac-a9d3-69a446c064c0.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00071-2bf5a32f-7612-41cf-b627-cf880e151737.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00081-c1a12de4-ba47-422c-91e2-12a31b3b66df.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00092-7ac21f42-792e-4a36-9ee1-443caf498f02.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00106-3607ae76-6936-4026-9bb2-abf13cfd2c31.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00122-a9a64285-15d5-42a3-8510-7395bcfb8e01.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00129-0f916586-0eeb-459c-aa86-e643d6331a76.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00132-d554b603-1b03-4328-bb4d-d8d03a56692e.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00140-60b4b97c-dd6a-46a8-a4f1-d6cff24c9457.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00147-e07443f9-d37b-40b9-a3bf-adb33ca81494.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00148-6b3821ab-6ff7-4422-a0d8-7ab3df540667.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00161-59a31e10-41ec-4ac5-94d4-1d522b4cbe7b.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00164-c53e81aa-c5c3-457d-ae75-d67c2d9d286d.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00169-55cb6295-b57f-46b3-ab0f-4fa759dec4fa.c000.snappy.parquet.crc
-a----	7/25/2026	4:12 PM	24	.part-00172-520abe8a-9d88-4713-a6dc-0fc146119616.c000.snappy.parquet.crc
-a----	7/25/2026	4:13 PM	24	.part-00182-4b5fac12-69dc-46c6-9b23-8ba2b7b860c9.c000.snappy.parquet.crc

Fig C.8: Parquet output confirming the data created on the disk

ward_id=W09	7/25/2026 4:11 PM	File folder
ward_id=W10	7/25/2026 4:11 PM	File folder
ward_id=W11	7/25/2026 4:11 PM	File folder
ward_id=W12	7/25/2026 4:11 PM	File folder
ward_id=W13	7/25/2026 4:11 PM	File folder
ward_id=W14	7/25/2026 4:11 PM	File folder
ward_id=W15	7/25/2026 4:11 PM	File folder
ward_id=W16	7/25/2026 4:11 PM	File folder
ward_id=W17	7/25/2026 4:11 PM	File folder
ward_id=W18	7/25/2026 4:11 PM	File folder
ward_id=W19	7/25/2026 4:11 PM	File folder
ward_id=W20	7/25/2026 4:11 PM	File folder
ward_id=W21	7/25/2026 4:11 PM	File folder
ward_id=W22	7/25/2026 4:11 PM	File folder
ward_id=W23	7/25/2026 4:11 PM	File folder
ward_id=W24	7/25/2026 4:11 PM	File folder

Task_C > output > ward_energy_summary_parquet > ward_id=W02 >			
Name	Date modified	Type	Size
date=2026-07-01	7/25/2026 4:13 PM	File folder	

Fig C.9: Data partitioned on disk by ward_id and date

C.3 Spark Structured SQL - Zone Health Advisories

Implemented in `spark/health_advisory_streaming_sql.py` as a Streaming SQL query (`spark.sql` over a streaming view) on `urbanpulse.air_quality`, completing all three following sub-parts.

- 10-minute rolling average AQI per zone, implemented as a sliding window (10-minute width, 1-minute slide) so the average updates every minute rather than only at fixed 10-minute boundaries.
- Joined with the static `zone_profile` reference table to enrich each advisory with `zone_name`, `population`, and `num_schools`.
- Filtered to `rolling_avg_aqi > 150` (Unhealthy) and written to **`urbanpulse.health_advisories`** in Update output mode, so an in-flight window's average is revised as new readings arrive.

Verified output: every emitted row exceeds the 150 threshold and carries the full zone enrichment:

```
{
  "zone": "Z03",
  "zone_name": "Bandra",
  "population": 297254,
  "num_schools": 4,
  "window_start": "2026-07-01T16:58:00.000+05:30",
  "window_end": "2026-07-01T17:08:00.000+05:30",
  "rolling_avg_aqi": 170.35
}
{"zone": "Z10", "zone_name": "Goregaon", "population": 389565, "num_schools": 11, "window_start": "2026-07-01T12:36:00.000+05:30", "window_end": "2026-07-01T12:46:00.000+05:30", "rolling_avg_aqi": 167.2}
{"zone": "Z11", "zone_name": "Juhu", "population": 266002, "num_schools": 25, "window_start": "2026-07-01T15:03:00.000+05:30", "window_end": "2026-07-01T15:13:00.000+05:30", "rolling_avg_aqi": 165.3}
{"zone": "Z18", "zone_name": "Worli", "population": 170819, "num_schools": 25, "window_start": "2026-07-01T15:25:00.000+05:30", "window_end": "2026-07-01T15:35:00.000+05:30", "rolling_avg_aqi": 169.1}
{"zone": "Z14", "zone_name": "Malad", "population": 419391, "num_schools": 22, "window_start": "2026-07-01T17:33:00.000+05:30", "window_end": "2026-07-01T17:43:00.000+05:30", "rolling_avg_aqi": 150.9}
```

Fig C.10: Sample messages created in `urbanpulse.health_advisories`

Offset	Partition	Timestamp	Key	Value
200	1	7/25/2026, 16:55:32	Z02	{\"zone\": \"Z02\", \"zone_name\": \"Andheri West\", \"popul...
201	1	7/25/2026, 16:55:32	Z02	{\"zone\": \"Z02\", \"zone_name\": \"Andheri West\", \"popul...
335	2	7/25/2026, 16:55:32	Z03	{\"zone\": \"Z03\", \"zone_name\": \"Bandra\", \"population\":...
336	2	7/25/2026, 16:55:32	Z10	{\"zone\": \"Z10\", \"zone_name\": \"Goregaon\", \"populatio...
337	2	7/25/2026, 16:55:32	Z11	{\"zone\": \"Z11\", \"zone_name\": \"Juhu\", \"population\": 26...
338	2	7/25/2026, 16:55:32	Z18	{\"zone\": \"Z18\", \"zone_name\": \"Worli\", \"population\": 17...
202	1	7/25/2026, 16:55:32	Z02	{\"zone\": \"Z02\", \"zone_name\": \"Andheri West\", \"popul...
203	1	7/25/2026, 16:55:32	Z08	{\"zone\": \"Z08\", \"zone_name\": \"Dahisar\", \"population\":...
204	1	7/25/2026, 16:55:32	Z12	{\"zone\": \"Z12\", \"zone_name\": \"Kandivalli\", \"population\"...
205	1	7/25/2026, 16:55:32	Z06	{\"zone\": \"Z06\", \"zone_name\": \"Colaba\", \"population\":...
339	2	7/25/2026, 16:55:32	Z14	{\"zone\": \"Z14\", \"zone_name\": \"Malad\", \"population\": 4...
206	1	7/25/2026, 16:55:32	Z12	{\"zone\": \"Z12\", \"zone_name\": \"Kandivalli\", \"population\"...
340	2	7/25/2026, 16:55:32	Z11	{\"zone\": \"Z11\", \"zone_name\": \"Juhu\", \"population\": 26...
318	0	7/25/2026, 16:55:32	Z13	{\"zone\": \"Z13\", \"zone_name\": \"Kurla\", \"population\": 4...
341	2	7/25/2026, 16:55:32	Z11	{\"zone\": \"Z11\", \"zone_name\": \"Juhu\", \"population\": 26...

Fig C.11: Screenshot of `urbanpulse.health_advisories` Kafka Topic

Offset	Partition	Timestamp	Key	Value
200	1	7/25/2026, 16:55:32	Z02	{\"zone\": \"Z02\", \"zone_name\": \"Andheri West\", \"popul...
201	1	7/25/2026, 16:55:32	Z02	{\"zone\": \"Z02\", \"zone_name\": \"Andheri West\", \"popul...

Fig C.12: filtering for `rolling_avg_aqi > 150` (Unhealthy) and landing in `Urbanpulse.health_advisories`

C.4 Flink vs Spark - UrbanPulse Use-Case Mapping

UrbanPulse deliberately uses both engines because they were built for opposite points on the latency/state spectrum not as redundant tooling. Comparing them against two of UrbanPulse's own use cases makes the split concrete rather than generic.

Dimension	Bus bunching - Flink	Ward energy - Spark
State size	Small, per-key: Bunching state is small and strictly local: per `route_id`, Flink only needs the latest position of each active bus on that route (~a few dozen buses per route, max) plus a short-lived proximity timer per bus pair. This is exactly what Flink's keyed state (`MapState`) is optimized for small, per-key, continuously mutated, and never needing a full-dataset scan to update.	Larger but compact: Aggregating total_kwh_consumed, avg_power_factor, and peak_voltage per ward_id per 15-minute window means state scales with (number of wards) × (open windows) large in absolute terms (1.1M meters feed into ~tens of wards), but the state itself is a compact per-ward accumulator, not per-meter. Spark's windowed aggregation over a DataFrame is built precisely for this shape: bulk, columnar, set-oriented computation.
Latency requirement	Sub-minute, per-event: Bunching must be detected while it's still happening i.e., the whole point is catching commuters a live ETA problem, not reporting it after the fact. Flink's per-event processing model evaluates each `bus_gps` reading the instant it arrives, so a bunching condition that crosses the 5-minute threshold is caught within the next event, not at the next micro-batch boundary.	Minutes: None of UrbanPulse's SLAs (sub-2-min AQI, 90s signal adaptation) apply to ward energy. It explicitly feeds 15-minute councillor dashboards and the 365-day regulatory audit trail. A few seconds to a minute of micro-batch latency is invisible at that reporting cadence.
Recovery time objective	Fast: Flink checkpoints the exact keyed state (Positions, proximity timers) incrementally and restores it on failure, so a restart resumes mid-stream with no re-derivation needed. RTO is bounded by checkpoint interval, typically seconds.	Slower but acceptable: Spark Structured Streaming recovers via checkpointed offsets + write-ahead state , replaying the last incomplete micro-batch from Kafka. Because windows are 15 minutes wide and the downstream consumers aren't real-time, a restart that takes longer than Flink's (re-reading a batch of Kafka offsets, rebuilding the last window's partial aggregate) is operationally acceptable here in a way it would not be for bus bunching.
Operational complexity	Higher: a standalone JobManager/TaskManager cluster, checkpoint storage, and (for the "real" Kafka Streams-style DSL UrbanPulse also uses in Task B) JVM tuning. This cost buys the low-latency, per-event guarantee that bunching detection actually needs.	Comparable cluster cost: Comparable cluster-ops burden to Flink (a Spark cluster + checkpoint storage), but the development complexity is lower for this specific job: groupBy(window(...)) plus a join against the static `zone_profile`/`route_schedule`-style reference table is idiomatic Spark SQL, whereas replicating the same 15-min tumbling aggregation in Flink would mean hand-rolling a window assigner and trigger for something Spark gives for free.

Flink is the correct choice for bus bunching because its **per-event, keyed-state model is the only way** to reliably detect a continuous 5-minute proximity condition. **Spark** is the correct choice for ward energy because the workload is **bulk and windowed rather than per-event**, and its latency budget comfortably absorbs micro-batch processing.

Live Monitoring Dashboard - Serving Layer Example

A lightweight Streamlit dashboard was developed to demonstrate the serving layer by consuming the following three output topics directly from Kafka using **confluent-kafka**. The dashboard displays incident alerts, ward energy summaries, and health advisories in near real time with automatic updates.

- **urbanpulse.incidents**
- **urbanpulse.ward_energy_summary**
- **urbanpulse.health_advisories**

The dashboard demonstrates that the outputs generated by Flink and Spark are successfully consumed by a downstream application, validating the end-to-end streaming pipeline proposed in Task A. It also served as a practical tool for verifying pipeline correctness by providing a unified view of all three output streams during development.



Fig C.13: Sample Live Serving Layer (Dashboard) highlighting the incidents or presenting aggregates for quick decision making
